

Faster Exact Unknot Recognition

Bounded skein filters, sparse cancellation, and factorwise homology

Implementation and mathematical analysis of the supplied `fast/` package

18 September 2026

Abstract

We provide an executable replacement for the Python unknot recognizer in the supplied archive. The implementation combines one-sided exact modular obstructions with a complete scanning Khovanov backend. Its improvements include a frontier Kauffman-bracket evaluation, a sparse modular determinant, an $O(n \log n)$ implementation of the existing scan-order heuristic, linear-time descending-diagram detection, cached cobordism operations, cheaper cancellation, and certified diagrammatic connected-sum factorization. Same-machine measurements show 45.8-fold and 53.9-fold recognition speedups on the Conway and Kinoshita–Terasaka examples, respectively; raw Conway homology improves by 2.82-fold. Eight trefoil summands exhibit a 722-fold raw-homology speedup.

A general $n^{O(\log n)}$ recognizer is *not* obtained. The unrestricted worst-case bound remains $2^{O(n)}$. There is, however, a rigorous factor-size bound $\text{poly}(n) + \sum_i 2^{O(n_i)}$ for the homology backend. It yields polynomial and quasi-polynomial bounds on appropriate restricted classes and avoids an exponential basis-size lower bound of the original representation. We explain why neither a small scan frontier nor Lackenby’s hierarchy iteration count alone justifies a general quasi-polynomial claim. The distribution includes source, 47 passing test methods, independent small-instance checks, unchanged baseline source, reproducible benchmarks, and all article sources.

Contents

1	Scope and baseline	2	7	Connected-sum factors	8
2	Input and correctness	2	8	Quasi-polynomial target	11
3	Modular determinant	3	9	Experiments	12
4	Frontier Jones evaluation	4	10	Validation	14
5	Faster preprocessing	6	11	Usage and limitations	15
6	Scanning homology	7	12	Conclusion	17

1 Scope, result, and the actual baseline

The input archive’s `fast/fastunknot/` is a complete recognizer, not an implementation of Lackenby’s hierarchical multi-surface algorithm. It first removes available Reidemeister I and II moves, checks whether the remaining diagram is descending, and computes an exact Alexander polynomial. When these stages do not decide the problem, it scans the diagram in the dotted-cobordism category over $\mathbb{F}_2$, cancels invertible differential entries, and reads the final Khovanov rank. The scanning strategy is of Bar-Natan type [4]; the implementation being optimized is the supplied source [1], not an external computer algebra package.

There are two different engineering targets. *Recognition* asks only for a verdict and should avoid unnecessary homology. *Raw homology* asks for the rank even when a much cheaper invariant already proves that the knot is nontrivial. Improving the latter is important for the fallback but must not be reported as an equal improvement in the former. All experimental tables below keep these workloads separate.

The delivered package is `fastunknot` version 0.2.0. Its runtime requires only Python’s standard library. The seven original Python files are retained, byte-for-byte, under `baseline_fastunknot/`; the containing directory is renamed so that the two versions can be imported independently. The ten original examples are unchanged. The accompanying `PROVENANCE.md` identifies every additional source, test, and measurement file.

Meaning of exact. All mathematical computations use integers, finite fields, or combinatorial objects, not floating-point approximations to a knot invariant. An executed program can nevertheless contain an error. The correctness arguments below justify the algorithms; the regression suite gives substantial finite evidence about their implementation. Neither the Python program nor this article has been formally verified in a proof assistant.

2 Input model and recognition contract

A diagram with $n > 0$ crossings is represented by a planar-diagram code $D = ((a_i, b_i, c_i, d_i))_{i=0}^{n-1}$. Ports occur counterclockwise; the under-strand joins ports 0 and 2, and the over-strand joins ports 1 and 3. Each edge label occurs twice. Labels are normalized to $0, \dots, 2n - 1$. The validator checks the single-component condition and the spherical rotation system, including the Euler face count $n + 2$. Thus it does not silently interpret a virtual diagram or a multi-component link as a classical knot. The exceptional code with no rows means one crossing-free circle.

Braid closures and rectangular diagrams are converted to this representation. Here and in all asymptotic statements, n is the number of crossings in the explicit converted PD code, *not* the side length of a grid. A grid may expand to quadratically many crossings. Bounds for normalized word-sized labels acquire the usual input-reading and integer-bit costs for arbitrarily long original labels.

The top-level algorithm is the following. “Different” always means different from the value forced by the unknot, not merely nonzero.

```

recognize(D):
    validate D
    apply crossing-decreasing R1/R2 moves
    if D has no crossings: return UNKNOT
    if D is descending from some basepoint: return UNKNOT
    if Delta_D(-1) modulo p differs from both +1 and -1:
        return KNOTTED
    attempt a bounded normalized Jones evaluation modulo p
    if its completed residue differs from 1: return KNOTTED
    if the full normalized Alexander polynomial differs from 1:
        return KNOTTED
    compute exact reduced F2 Khovanov rank, using visible factors
    return UNKNOT exactly when that rank equals 1

```

The Jones stage’s own state or transition ceiling only skips that optional filter. It does not produce UNKNOWN. A user-selected global time budget or homology object ceiling can produce UNKNOWN, never an inferred knot verdict. With no such global ceilings, the fallback remains complete.

Proposition 2.1 (Sound decision rule). *Assuming the mathematical local operations are implemented correctly, every completed verdict of the pipeline is correct. Without a global resource limit, the pipeline terminates on every valid input.*

Proof. Reidemeister moves preserve knot type, and a descending knot diagram represents the unknot: lift each successively traversed overpass above the untraversed part, then straighten the resulting descending arc and close it. The invariant filters are justified in Sections 3 and 4; none can reject an unknot. A nonconstant normalized Alexander polynomial also excludes the unknot.

Kronheimer–Mrowka’s unknot-detection theorem says that reduced Khovanov rank one over $\mathbb{Q}$ characterizes the unknot [5]. The integral reduced chain complex is finite and free. Universal coefficients imply

$$\dim_{\mathbb{F}_2} \widehat{\text{Kh}}(K; \mathbb{F}_2) \geq \dim_{\mathbb{Q}} \widehat{\text{Kh}}(K; \mathbb{Q}).$$

The rational rank is nonzero (its graded Euler characteristic is the normalized Jones polynomial, whose value at 1 is 1). Thus reduced $\mathbb{F}_2$ rank one implies the unknot, while the unknot has reduced rank one directly. Sections 6 and 7 justify the backend transformations. The finite cube of resolutions gives termination, and all preceding stages have finite work or an explicit filter ceiling. $\square$

3 A sparse modular determinant obstruction

The existing Alexander routine constructs a Wirtinger matrix over $\mathbb{Z}[t]$ and performs fraction-free polynomial elimination. That can be more work than is needed to exclude the unknot. At $t = -1$, the row associated with either oriented crossing has the simple form

$$2e_{\text{over}} - e_{\text{under,in}} - e_{\text{under,out}}. \tag{1}$$

The over-arcs are obtained by joining the two over ports at every crossing. After deleting a row and a column, the determinant is $\Delta_K(-1)$ up to sign: any monomial unit $\pm t^j$ becomes a sign at $t = -1$. This is the same Alexander presentation used by the original code, specialized before elimination.

The new routine builds sparse row dictionaries and computes this minor modulo $p = 1\,000\,000\,007$. At each pivot column it selects a nonzero pivot row of minimum current support, swaps it into

position, multiplies the accumulated determinant by the pivot, and eliminates only rows with a nonzero entry in that column. It returns the resulting residue, including zero for a singular minor.

Lemma 3.1. *If this residue is not in $\{1, p-1\}$, the input knot is nontrivial.*

Proof. For the unknot, every relevant normalized Alexander minor specializes to ± 1 , so its residue belongs to that set. The contrapositive proves the claim. No converse is asserted: an integer determinant congruent to ± 1 can be much larger in absolute value, and determinant one itself does not characterize the unknot. $\square$

The sparse representation does not improve the dense worst-case $O(n^3)$ field operation bound, but it avoids coefficient-polynomial growth and often limits fill. Field elements have a fixed 30-bit modulus. This is a deterministic one-sided obstruction, not a Monte Carlo recognition rule. The prime and the evaluation point need not be randomly selected for soundness.

4 Bounded frontier Jones evaluation

4.1 State and transfer

For the PD convention above, smoothing 0 joins $(0, 1), (2, 3)$ and has weight A ; smoothing 1 joins $(0, 3), (1, 2)$ and has weight A^{-1} . Set

$$\delta = -A^2 - A^{-2}.$$

The normalized Kauffman bracket is a presentation of the Jones invariant [2]. The implementation evaluates it in a residue ring where both A and δ are invertible. The default is $A = 2$ modulo p , but the Python API supports other valid moduli and units.

At a partial scan, the boundary consists of diagram edges occurring in exactly one processed crossing. A state is a perfect matching m of those boundary labels, represented by a sorted tuple of sorted endpoint pairs. A dictionary stores one coefficient $c(m)$ per matching. Closed circles are not retained as geometric objects: their scalar factor δ is multiplied in immediately.

For each old state and the two smoothings, join the matching arcs to the local smoothing arcs. The result has a new matching m' and ℓ newly closed circles. The transition is

$$c_{\text{new}}(m') += c_{\text{old}}(m)A^{1-2s}\delta^\ell, \quad s \in \{0, 1\}. \quad (2)$$

All arithmetic is reduced modulo the chosen modulus, and zero entries are removed. A mate dictionary implements arc joining without invoking the homology scanner's `glue` function. At one crossing $\ell \leq 2$, so the three possible powers of δ can be prepared once.

Lemma 4.1 (Transfer invariant). *After any set of crossings in the chosen scan order has been processed, $c(m)$ equals the sum of the weights of all partial smoothing states that induce m , with one factor of δ for each already closed component.*

Proof. Initially the only state is the empty matching with coefficient 1. Every partial smoothing extends uniquely by choosing smoothing 0 or 1 at the next crossing. Arc joining identifies exactly its boundary connectivity, and exactly its newly closed components disappear from that connectivity. Equation (2) therefore adds each extension once with the correct weight. Induction proves the statement independently of the order of the crossings. $\square$

4.2 Normalization and orientation

At the end the only possible matching is empty. The computed scalar is

$$B_D(A) = \sum_{s \in \{0,1\}^n} A^{n-2|s|} \delta^{c(s)},$$

where $c(s)$ is the number of circles in the complete smoothing. For $n > 0$ the normalized value used for the obstruction is

$$J_D(A) = (-A^3)^{-w(D)} \delta^{-1} B_D(A). \quad (3)$$

It equals $V_K(A^{-4})$ with the package's consistent choice of crossing sign and variable. In particular, its value on the unknot is 1. The crossing-free input is handled directly as 1, rather than applying a nonempty-diagram formula to a different convention for the empty link.

There is a relevant orientation issue in the original code. Computing a sign from the incoming over port alone assumes that the incoming under port is always slot 0. A legitimate half-turn of an individual PD row need not preserve that assumption. The new implementation determines both incoming ports from an oriented traversal. It uses

$$\epsilon_i = +1 \iff (o_i - u_i) \bmod 4 = 3, \quad w(D) = \sum_i \epsilon_i.$$

This is invariant under half-turns and orientation reversal. The Alexander routine also uses the actual incoming under port. We do not infer from this change that the original recognizer gave wrong verdicts: compensating choices in its Alexander presentation can preserve the polynomial. It is essential, however, not to reuse an inconsistent writhe in a new Jones filter. Under this internal convention the positive braid generator used by the converter has negative signed writhe; users should not mix it with a different convention for formula (3).

Proposition 4.2 (Jones obstruction). *If a completed evaluation gives $J_D(A) \neq 1$, the input is not the unknot. An evaluation congruent to 1, or an interrupted evaluation, gives no verdict.*

Proof. The bracket relations, including writhe correction at Reidemeister I, give the same normalized value on isotopic oriented knots. Evaluation in a ring where the displayed inverses exist respects these relations. Hence an unknot must have residue 1. This proves only the stated implication. $\square$

For example, the Conway and Kinoshita–Terasaka files both have Alexander polynomial 1 in the tests, but the default Jones residue is $619\,646\,344 \neq 1$. At most five frontier states suffice for these scans. The suite also deliberately evaluates a trefoil at $A = 1$ modulo 5: its value then equals the unknot's value. The program treats this collision as inconclusive, not as evidence of triviality.

4.3 State count and budget

Let w be the maximum number of boundary endpoints and S the maximum number of matchings retained at a step. Each complete partial resolution induces one matching, so a safe count is

$$S \leq \min\{2^n, (w-1)!!\}, \quad (w-1)!! = \frac{w!}{2^{w/2}(w/2)!} \quad (4)$$

for positive even w ; the count for $w = 0$ is one. One must not automatically replace this by a Catalan number: an arbitrary processed region need not be a single disk with all endpoints on one ordered boundary component.

There are at most $2nS$ transfers. Arc joining and canonical sorting cost $O((w+1)\log(w+2))$ word operations per transfer. With the usual expected constant-time dictionary model this gives

$$O(nS(w+1)\log(w+2) + n\log n).$$

For a deterministic comparison dictionary, include an additional logarithm of the state count; adversarial hashing can also worsen the dictionary factor. None of the global correctness or exponential upper bounds depend on an expected-time hash assumption.

The default filter ceilings are 8192 live keys and 250000 transitions. Thus this optional stage has polynomial overhead even on diagrams where the unbounded bracket computation is expensive. Both old and new dictionaries may be live during a transition; the key ceiling is not a promise about total bytes. Inconclusive residues and filter-limit exceptions go to the unchanged mathematical fallback, not to a heuristic verdict.

5 Two provably faster preprocessing subroutines

5.1 The same greedy scan policy, with local updates

The original heuristic tries several first crossings. At each subsequent step it scans every unprocessed crossing, favoring more shared boundary edges and then a smaller boundary increment. On a connected four-valent diagram, choosing one crossing changes the priority of at most four distinct neighboring crossings. Parallel edges may produce several increments to the same neighbor, but the total number of incidences remains $O(n)$.

The new routine stores priorities in a lazy heap and pushes updated priorities only for affected neighbors. Entries for already chosen crossings or old shared counts are discarded on extraction. The tie-breaker is the crossing index, as in the original implementation. The loop contribution is fixed per crossing, so checking the current shared count also checks the changing part of its key. Consequently this is an implementation of the same greedy rule, not a different heuristic whose quality might confound the timing comparison.

Proposition 5.1. *For a fixed number of starting crossings, the new ordering implementation uses $O(n\log n)$ word operations and $O(n)$ storage, instead of the original $O(n^2)$ per fixed number of starts.*

Proof. The heap begins with n entries. Across a scan, each edge causes only a constant number of updates, so there are $O(n)$ pushes and pops. Each costs $O(\log n)$. Boundary-profile evaluation is linear, and a constant number of candidate scans does not change the bound. $\square$

The production call uses the same bounded-start policy as before. Calling `best_scan_order` with all n starts instead gives an extra factor of n . The API's `tries` parameter is a sampling target, not necessarily the exact number of starts; integer spacing can produce more than that many, but still only a constant number when the target is fixed.

5.2 Linear descending-basepoint detection

An oriented knot traversal has $2n$ crossing visits, each marked over or under. For one basepoint, count crossings whose first visit is under. This gives a nonnegative “bad” count. Moving the

basepoint across an over visit makes that crossing's under visit come first and increases the count by one; moving across an under visit decreases it by one. No other crossing changes its first visit.

After an initial linear traversal, one cyclic sweep therefore finds every zero-bad-count basepoint in linear time. Repeat for the reverse orientation and choose the smallest valid incoming dart, matching the original return-value convention. The original code retraverses the knot for each candidate dart and is quadratic. R1/R2 reduction itself has not been replaced by an elaborate new simplifier: repeated rebuilding and validation can still be quadratic, up to label-sorting factors. Claims about the descending subroutine do not remove that separate cost.

6 Faster exact scanning homology

6.1 Representation and cancellation

The original scanner already applies the main local-computation idea: process one crossing, deloop newly closed circles into two summands, and cancel invertible entries before adding the next crossing. At a frontier, an object is a boundary matching together with a cube degree. Several copies of the same matching can survive. A morphism is a finite $\mathbb{F}_2$ -linear combination of dot subsets on the circles obtained by gluing its source and target matchings. The Frobenius algebra underlying the local theory is

$$A = \mathbb{F}_2[x]/(x^2), \quad \Delta(1) = 1 \otimes x + x \otimes 1, \quad \Delta(x) = x \otimes x,$$

with $\epsilon(1) = 0$ and $\epsilon(x) = 1$. This is the characteristic-two specialization of the usual Khovanov construction [3].

Suppose a differential entry $\phi : B \rightarrow C$ is invertible. For every other incoming arrow $\delta : X \rightarrow C$ and outgoing arrow $\gamma : B \rightarrow Y$, deleting B, C replaces the $X \rightarrow Y$ entry by

$$d_{XY} + \gamma\phi^{-1}\delta. \tag{5}$$

This is chain-level Gaussian elimination: change bases to isolate the contractible two-term complex $B \xrightarrow{\phi} C$ and remove it. Over $\mathbb{F}_2$ the usual subtraction is addition. It preserves chain homotopy type, not just an Euler characteristic. The final closed minimal complex has zero differential, and the number of its objects is its homology dimension.

6.2 Every relevant unit is its own inverse

For a matching with r arcs, its endomorphism algebra in the stored normal form is

$$R_r = \mathbb{F}_2[x_1, \dots, x_r]/(x_1^2, \dots, x_r^2).$$

A dot subset corresponds to the square-free monomial in the associated arc variables. Multiplication of two such monomials is zero when their dot sets intersect, and is their union otherwise.

Lemma 6.1. *Every unit of R_r is of the form $1 + \nu$, with ν having no constant term, and satisfies $(1 + \nu)^{-1} = 1 + \nu$.*

Proof. The ideal generated by the variables is nilpotent, and the quotient is $\mathbb{F}_2$; thus an element is invertible exactly when its constant coefficient is one. In characteristic two, squaring a sum has no mixed terms. Every nonconstant square-free monomial has square zero. Therefore $\nu^2 = 0$ and $(1 + \nu)^2 = 1$. $\square$

The baseline computes the inverse through a geometric-series procedure. The replacement simply copies the unit's coefficient set. It also short-circuits compositions with identity and uses dot-set multiplication when source, intermediate, and target matchings all coincide. These are exact algebraic identities, not approximation thresholds.

6.3 Cache topology separately from coefficients

For a composition $a \rightarrow b \rightarrow c$, the connected components and Euler characteristics of the glued surface depend on (a, b, c) , not on the coefficient sets of the two morphisms. The new `_compose_plan` computes this topology once and caches it. Dot counts are then accumulated against the plan and the surface evaluated in normal form. A second cache memoizes immutable completed compositions. Crossing-transfer entries are likewise cached using immutable keys, and callers receive owned coefficient sets so ordinary mutation cannot corrupt those entries.

The cache sizes are explicit: 8192 entries each for circle data, gluing, composition topology, and crossing entries; 16384 for completed compositions. A raw scan clears these caches both before computation and in a `finally` block afterward. Repeated-factor reuse is a separate per-call PD memo, not an unbounded cross-call cache. Entry counts do not bound the sizes of individual cached values, so this is not a hard memory guarantee.

6.4 Sparse-fill cancellation priorities

If a candidate unit $B \rightarrow C$ has u other arrows into C and v other arrows out of B , formula (5) can require uv updates. The new cancellation queue favors smaller uv . It maintains lazy priorities, refreshes affected neighbors, and rebuilds the queue when stale entries dominate. The cost is an estimate: it does not account fully for morphism support or guarantee a globally minimum-fill elimination order.

Every selected arrow must still be a unit between equal matchings. Thus changing priorities changes performance, not the homotopy argument. Deadline checks are also inserted into expensive loops. The updated composition statistic counts actual calls, whereas the old code counted some reused half-compositions more than once; raw composition counters should not be interpreted as a perfectly matched instruction metric across versions.

7 Certified connected-sum factorization

7.1 Finding cuts in the input diagram

Write down the cyclic sequence of the $2n$ crossing visits in an oriented traversal. Each crossing label appears twice. A proper interval in which every crossing appears either zero or twice identifies a set S of whole crossings. Its complement contains the remaining crossings. A sweep toggles crossing labels in an open set; that set is empty exactly at a complete-crossing interval. Only intervals of length at most n need be considered, because longer ones have shorter complements. Among candidates the implementation favors a split with a larger smaller side.

Lemma 7.1 (Two-edge-cut criterion). *Such a nonempty proper interval in a spherical one-component knot diagram provides a diagrammatic connected sum. Closing each side by joining the two cut endpoints gives valid factor diagrams.*

Proof. The interval is a single traversal segment, so the subgraph on its crossing set is connected; the complementary segment shows the same for the complement. Exactly the two edges at the ends of the segment join the two sides. Since both sides are connected, this pair is a minimal edge cut, or bond, in the plane multigraph. The dual edges of a bond form a cycle. In the two-edge case this is a simple closed curve through two complementary regions, transverse to precisely the two original edges and disjoint from the vertices. Its two disks give the usual planar connected-sum decomposition. Joining the endpoints in each disk closes the corresponding factor without adding a crossing. $\square$

The implementation does not trust the interval search alone: it recounts the boundary edges and requires exactly two. It then closes each side by identifying those two edge labels and calls the diagram validator again. Recursion stops when this particular diagram has no further detected cut. This is *not* a claim that the knot is prime; an isotopy might reveal a connected sum invisible in the present projection. A factor can also represent the unknot.

A split search takes $O(n^2)$ word operations, apart from normalizing child labels. There are at most $n-1$ splits, giving a conservative polynomial $O(n^3)$ search bound, with ordinary label-operation factors understood. The recursive result includes the local cut labels, the two crossing-index sets, and the child certificates. The supplied replay function validates these partitions and boundaries and reconstructs the factors. Labels at a certificate node refer to that node's normalized PD, not to the original root diagram. Replay verifies cuts, not the homology calculation on a leaf.

7.2 Why reduced ranks multiply over $\mathbb{F}_2$

For completeness, the algebra needed by the implementation can be derived directly from the marked Frobenius complex. Mark a point on the knot away from crossings. On each resolution, let X multiply the marked circle factor by x . Its image is a reduced complex $\tilde{C}$, up to the irrelevant quantum shift. Both X and the differential are A -linear with respect to this marked action.

There is a useful characteristic-two splitting. Let ν be the sum, over all circle factors in a resolution, of the map ∂ with $\partial(1) = 0$ and $\partial(x) = 1$. The equations

$$\partial m = m(\partial \otimes 1 + 1 \otimes \partial), \quad \Delta \partial = (\partial \otimes 1 + 1 \otimes \partial) \Delta$$

follow by checking $1, x$ and their tensor products. Hence ν commutes with the cube differential. On every marked resolution,

$$X\nu + \nu X = 1. \tag{6}$$

The terms differentiating unmarked circles occur twice and cancel; on the marked factor this is the elementary identity $x\partial + \partial x = 1$.

Proposition 7.2. *After forgetting quantum grading, the unreduced knot complex over $\mathbb{F}_2$ splits as two copies of its reduced complex with the same cube degrees. In particular, its rank in each cube degree is twice the reduced rank in that degree.*

Proof. At chain level, $\ker X = \operatorname{im} X = \tilde{C}$. The map $X : C \rightarrow \tilde{C}$ is surjective, and ν restricted to $\tilde{C}$ is a chain section by (6). Thus $C \cong \tilde{C} \oplus \tilde{C}$ as complexes when quantum shifts are ignored. The maps preserve cube degree. Taking homology proves the claim. $\square$

For a connected sum, resolutions are pairs of factor resolutions and the two marked circles are joined. The marked complex is the tensor product of the factor complexes over A . Reducing at

the joined circle is base change $A \rightarrow A/(x) = \mathbb{F}_2$, equivalently the image-of- X model above up to a quantum shift. Thus its reduced complex is the tensor product of the reduced factor complexes over $\mathbb{F}_2$. Since this is a field, the ordinary Künneth formula has no Tor terms. Iterating gives

$$\widetilde{\text{Kh}}(K_1 \# \cdots \# K_k; \mathbb{F}_2) \cong \bigotimes_{i=1}^k \widetilde{\text{Kh}}(K_i; \mathbb{F}_2). \quad (7)$$

If a raw factor scan returns unreduced degree counts $b_{i,h}$, the implementation first checks that they are even and forms

$$P_i(z) = \sum_h \frac{b_{i,h}}{2} z^h, \quad P(z) = \prod_i P_i(z). \quad (8)$$

The final reduced rank is $P(1)$ and the final unreduced degree counts are twice the coefficients of P . The code never forms a tensor-product basis simply to count its elements. Since closing a cut introduces no crossing, the local 0/1 cube degrees add without an extra shift. These are explicitly unnormalized cube degrees, not an assertion that a signed homological or quantum grading has been returned.

7.3 A genuine representation-level asymptotic improvement

Theorem 7.3 (Factor-size complexity). *Let the detected factors have crossing counts $n_1, \dots, n_k$, with $\sum_i n_i = n$, and let $m = \max_i n_i$. With no global budget, the exact rank backend has time bound*

$$\text{poly}(n) + \sum_{i=1}^k 2^{O(n_i)} \leq \text{poly}(n) + k2^{O(m)}. \quad (9)$$

Its memory can also be bounded by a polynomial plus exponential work in the largest factor. Identical-factor memoization can improve the sum but is not needed for these bounds.

Proof. Each factor has at most 2^{n_i} smoothing states and at most $n_i + 1$ circles per complete state. Its expanded Frobenius chain size is therefore $2^{O(n_i)}$. The local scanner, its morphism supports, and Gaussian elimination are bounded by a polynomial in exponential-size data; changing the order or caching does not require more than $2^{O(n_i)}$ time. The cut search is polynomial.

The product (8) has degree at most n . Every coefficient is bounded by the full cube dimension $2^{O(n)}$ and thus has $O(n)$ bits. Repeated ordinary integer convolution consequently has polynomial bit complexity (a loose schoolbook bound suffices). It stores counts, not basis elements. The factor scans can be run sequentially, retaining only polynomial-sized degree reports; the per-call memo does the same for repeated factors. $\square$

Corollary 7.4. *On the class of diagrams for which the detected largest factor has $m = O(\log n)$, the backend is polynomial-time. For $m = O((\log n)^2)$ it has a quasi-polynomial bound $n^{O(\log n)}$. The complete recognition pipeline obeys these bounds as well, since its preceding stages have polynomial cost under the default fixed Jones ceilings.*

For a concrete family, let D_k be the explicit connected sum of k copies of the three-crossing trefoil supplied by the braid converter. Its reduced rank is 3^k and its unreduced rank is $2 \cdot 3^k$. In the code's cube convention the computed degree-count polynomial is

$$2(1 + z + z^3)^k.$$

The single-trefoil result is also checked with the independent cube routine. The original raw scanner has one surviving object per unreduced homology generator, so its final representation alone costs $\Omega(3^k)$ space and time. The new routine uses small factor scans and polynomial-size integer degree counts. This is an asymptotic improvement on an infinite, explicitly generated family, not merely a timing observation.

It is *not* a lower bound for recognizing D_k . In fact, both default recognizers normally exclude these knots with a polynomial invariant before reaching homology. The improvement is to the full-rank backend and to its applicable parameterized upper bound.

8 Why this does not establish general quasi-polynomial time

8.1 What the supplied Lackenby materials establish

The supplied slides are dated February 2021 and state a quasi-polynomial unknot-recognition theorem. They outline hierarchical multi-surfaces and acceleration operations, rather than a Khovanov scanner [6]. The supplied 2026 paper is a distinct, detailed hierarchy-based algorithm. Section 9 bounds iterations by $L(g+1)^L$ and explicitly separates this from precise operation costs; Section 10 refines the procedure to control hierarchy length linearly in initial q -complexity [7]. It would therefore be inaccurate either to identify the unaccelerated iteration estimate with a bit-complexity bound, or to say that the paper contains no length refinement.

For orientation, a sufficient abstract acceleration contract would bound each compressed step by $\text{poly}(n)$, keep pattern complexity $g \leq n^c$, and ensure an appropriate effective depth $L \leq C \log(n+2)$. The elementary calculation

$$L(g+1)^L \text{poly}(n) = \exp(O((\log n)^2)) = n^{O(\log n)}$$

then follows. This calculation alone is not an implementation of the contract.

A working hierarchy port would need explicit, complexity-controlled representations for the manifold, boundary pattern, decomposing surfaces, and parallelity data; verified local operations for cuts, compressions, and simplifications; and a global progress bound preserved by every operation. Compressed coordinates must remain compressed during construction and updates, not only during certificate storage. The talk's named Cheeger-region and multi-surface operations cannot be implemented merely by declaring functions with those names. This package does not contain such hidden oracles or substitute normal-surface enumeration for an unimplemented fast step.

The development effort here instead implements exact accelerations that can be proved locally and measured against the given code. It does not resolve the remaining general hierarchy implementation and bit-complexity obligations. That is a limitation of this delivered solution, not a claim that Lackenby's announced algorithm is impossible or that an open problem cannot be attacked.

8.2 The frontier-width pitfall

A small boundary can bound the *types* of matchings but not their chain multiplicities. In a chain of trefoil summands, one can scan one small summand at a time across a bounded interface. Nevertheless the final full homology rank is $2 \cdot 3^k$. Even the final frontier is empty, with just one matching type, while the original scanner has exponentially many copies of it.

Consequently a statement of the form

$$\text{full scanning homology time} = \text{poly}(n) 2^{O(w)}$$

does not follow from frontier matching counts. A claimed separator argument must also control generator multiplicities, matrix ranks, and coefficient supports or represent their effect compactly. The scalar Jones DP does aggregate coefficients of identical matchings, so its state-count argument is different. The factor product does compress multiplicities in a provable special case. Neither observation gives such compression for every prime, indecomposable, or difficult unknot diagram.

8.3 Remaining unrestricted upper bound

The finite full cube supplies an exponential upper bound for the unchanged mathematical fallback. The added filters, search, and integer convolutions do not worsen its asymptotic class. Thus the report and the result JSON state $2^{O(n)}$, not $n^{O(\log n)}$, for unrestricted input. An experimentally small number of frontier states, a low rank on one example, or a faster constant in cancellation is not evidence of a universal complexity exponent.

9 Experiments: protocol and results

9.1 Protocol

The benchmark script runs the unchanged baseline and the accelerated package on the same input and machine in separate worker processes. The environment was Python 3.13.5 (GCC 14.2.0), Linux x86_64, on a system reporting an AMD EPYC 9V74 80-Core Processor. Each worker uses one Python process; no parallel speedup is claimed. `PYTHONHASHSEED=0` is set for reproducibility. Hardware sharing and scheduler noise are not controlled, so these are measurements in this environment rather than portable performance promises.

There are 65 case/task/implementation records. Successful records normally contain five timed samples; tables report their medians. Caches are cleared before each sample, and garbage collection is run outside the timed interval. Interpreter startup, imports, JSON parsing, and initial diagram construction are excluded. The new public routines' internal revalidation is included. Within-call factor memoization is part of the algorithm and remains enabled.

Both versions receive the same 200000-object ceiling and nominal eight-second cooperative budget per sample. A worker also has an external wall limit of $5 \cdot 8 + 3 = 43$ seconds for the entire five-sample job. Workers stop repeating when a structured UNKNOWN is returned. An externally killed job has no returned sample summary and is not converted into a per-sample runtime.

The corpus contains every supplied example, $T(3, q)$ braid closures for $q = 31, 61, 121$, a curl family, explicit connected sums, and the precise 36-crossing five-strand braid from the original archived benchmark. The families and orders are generated in `tools/benchmark.py`. No knot database lookup or expected answer is used by the recognizer.

9.2 End-to-end recognition

The Conway and Kinoshita–Terasaka examples exercise a useful distinction: Alexander is inconclusive, but a tiny scalar Jones scan excludes the unknot before homology. For $T(3, 61)$ the original full-polynomial stage is much more expensive than the new modular path, yielding the measured 521.5-fold speedup. This does not demonstrate a better general asymptotic class for the whole recognizer; it demonstrates the cost of computing more of an invariant than this decision needs.

The full table deliberately retains regressions. The eight-crossing hard unknot and several tiny

Table 1: Default recognition, milliseconds. All completed verdicts agree.

Input	n	Original	New	Original/new
Conway	11	33.964	0.742	45.77×
Figure eight	4	0.153	0.169	0.90×
Determinant-one grid	10	0.877	0.694	1.26×
Scrambled grid unknot	6	0.346	0.374	0.92×
Hard unknot (8 crossings)	8	1.826	1.880	0.97×
Kinoshita–Terasaka	11	38.334	0.712	53.86×
$T(3, 5)$	10	0.686	0.658	1.04×
Trefoil	3	0.140	0.167	0.84×
Small unknot	0	0.024	0.036	0.65×
40-crossing braid unknot	40	6.469	6.947	0.93×
$T(3, 31)$	62	287.726	3.518	81.79×
$T(3, 61)$	122	3,710.382	7.115	521.50×
$T(3, 121)$	242	<i>worker timeout</i>	16.104	—
36-crossing stress braid	36	8.053	2.095	3.84×

The $T(3, 121)$ original worker was stopped after 43.05 seconds for the entire job; no completed-sample summary was returned. This is not a measured single-run time. Small-input regressions are included.

examples are slightly slower in the default pipeline. Their inexpensive original path leaves little work to save, while revalidation and additional inconclusive filters have a nonzero cost. There is no claim that the new defaults dominate the old ones on every individual diagram.

9.3 Raw homology and the stress braid

Table 2: Raw unreduced homology, milliseconds; no recognition prefilters.

Input	n	Original	New, no factors	New
Conway	11	34.701	12.404	12.319
Kinoshita–Terasaka	11	38.677	14.115	14.071
Hard unknot (8 crossings)	8	2.300	1.897	1.964
40-crossing braid unknot	40	8.664	3.568	2.473
Curl family 16	16	2.223	—	0.967
Curl family 64	64	17.344	—	3.706
Curl family 256	256	229.778	—	20.516
36-crossing stress braid	36	<i>cutoff at</i> 8,037.1	1,697.750	1,766.619

The stress baseline cutoff is one unfinished sample, not a median completion. A separate 60-second baseline run also remained unfinished (60.10 seconds elapsed). Dashes mean not run. Factor detection can add small overhead when no cut exists.

The no-factor column isolates much of the local scanner improvement from cut search. On Conway it takes 12.40 milliseconds, versus the original 34.70; with the standard factor option it takes 12.32 milliseconds. On the stress braid the local backend finishes in 1.698 seconds without factor search and 1.767 seconds with the default factor stage. The result is unreduced rank 5898, with peak 17694 objects before a crossing’s elimination and boundary size 8. The difference between the two new timings is consistent with factor-search overhead and ordinary measurement noise, not evidence of a cut-based speedup on that input.

The original raw stress scan reached its eight-second cutoff. A separate single-sample run with a

60-second budget also returned UNKNOWN, after 60.10 seconds. A further attempt on the 24-crossing R2-reduced diagram reached a ten-second cutoff. These attempts and their failures are preserved in the results directory. We do not report an exact baseline completion-time ratio, and we have not independently established the large stress rank by completing an original scan. The small-instance independent tests and the completed paired comparisons provide the correctness evidence described below.

The same stress braid is recognized by the two default pipelines in 8.053 and 2.095 milliseconds, respectively. Substituting those recognition times for raw homology would hide the question the stress benchmark is intended to test.

9.4 Ordering and factorization

Table 3: Identical greedy ordering policy on the curl family, milliseconds.

n	Original	New	Original/new
64	14.822	1.477	10.03×
256	232.150	6.008	38.64×
1024	3,429.239	29.110	117.80×

Table 4: Raw homology of k trefoil summands, milliseconds.

k	n	Unreduced rank	Original	New	Original/new
2	6	18	2.365	0.964	2.45×
4	12	162	14.648	1.212	12.08×
6	18	1,458	128.899	1.511	85.28×
8	24	13,122	1,287.361	1.783	722.19×
16	48	$2 \cdot 3^{16}$	—	2.616	—
32	96	$2 \cdot 3^{32}$	—	5.736	—

Baseline runs at $k = 16, 32$ were not attempted. The new algorithm reports integer degree counts, not one basis vector per homology generator. These are raw-homology timings: both recognition pipelines can reject trefoil sums much earlier.

For the ordering experiment, the curl family is the closure of the braid on $n + 1$ strands with word $[1, 2, \dots, n]$. Only the choice of scan order is timed. At 1024 crossings the new implementation is 117.8 times faster while following the same greedy policy. This isolates an asymptotic subroutine improvement, rather than an easier knot invariant or a changed order.

For 32 trefoil summands, the returned total unreduced rank is 3 706 040 377 703 682. The new run takes 5.736 milliseconds, with at most 18 explicit objects in a factor scan and 32 factor reports. Identical normalized PDs are reused; the degree polynomial is still assembled by integer convolution. A direct basis-sized result would not even be a sensible output format at this rank. The polynomial-size numerical output is precisely the representation exploited by Theorem 7.3.

10 Validation and limits of the evidence

The final suite contains 47 test methods and passes in the recorded run. Nineteen are the original tests, with an explicit adapter disabling the two new filters where an old assertion requires the original method name; a matching CLI flag is added for that expectation. The assertions are

retained, rather than silently changing expected answers to whatever the new code returns. The remaining methods exercise the following independent or differential paths.

Scalar invariants. Eighty random small knot diagrams are compared against an independent full 2^n smoothing-state DSU calculation, at two moduli and with shuffled crossing orders. It does not use the frontier transfer routine. Examples of at most 11 crossings receive the same full-state check. Tests cover braid relations, inverse-pair insertion, conjugation, stabilization, mirrors, deliberately colliding residues, and resource ceilings. One hundred random diagrams compare the modular determinant with evaluation of the full polynomial.

Orientation and ordering. Sixty random diagrams undergo independent half-turns of PD rows; writhe, Alexander, and Jones values are checked for invariance. Eighty random diagrams compare the fast greedy order with the original for every possible first crossing. One hundred compare the linear descending test's exact returned dart with the original test. These checks target convention errors and tie-breaking, not just total running time.

Local algebra and homology. Five hundred randomly selected morphism compositions, spread across boundary sizes 0, 2, 4, 6, and 8, are compared with the original surface-composition routine. One hundred twenty random units are checked to square to identity using that original routine. Fifty-five random small raw scans with shuffled orders check $d^2 = 0$ after every crossing, compare degree distributions with the original scanner, and compare total reduced rank with a separately constructed cube complex. Eighty random recognition verdicts are also checked against that independent reduced cube. The independent cube uses the Frobenius multiplication and comultiplication directly; it does not use the optimized cancellation code.

Factors, API, and failure paths. Known and mixed connected sums test rank products and degree convolution, including mirrors, row permutations, and half-turns. Explicit scan orders are checked to bypass factorization. Cut-certificate replay is tested against the generated factors and against deliberately tampered records. Other tests cover cache clearing after success and failure, invalid orders and ceilings, global zero-time budgets, filter-cap fallback, and the new CLI options and exit codes.

The benchmark harness asserts agreement among completed paired verdicts and raw degree distributions; its JSON ends with `completed_results_agree: true`. This is not a claim of a result from an unfinished baseline job. Testing remains finite, and comparison with unchanged baseline code does not detect a bug shared by both implementations. The independent cube and bracket paths address that risk only on the small instances where they were actually executed.

11 Using and extending the solution

11.1 Commands and API

No installation is necessary: run from the distribution root with Python 3.10 or later. For example:

```
python -m fastunknot recognize examples/conway.json
python -m fastunknot recognize examples/hard_unknot_8.json
python -m fastunknot khovanov examples/conway.json --check-d2
python -m fastunknot jones examples/conway.json
python -m fastunknot recognize examples/conway.json \
    --max-jones-states 1
```

The last call deliberately makes the Jones filter inconclusive and exercises the exact fall-back. For an unfiltered homology-based recognition call, use `-no-reduction -no-descending -no-alexander -no-jones`.

The main Python entry points preserve their original names:

```
from fastunknot import Diagram, recognize, khovanov_rank
from fastunknot.jones import jones_residue
from fastunknot.factors import replay_factor_certificate

D = Diagram.from_braid(2, [1, 1, 1])
answer = recognize(D)
assert answer.status == "KNOTTED"
rank_data = khovanov_rank(D.pd)
assert rank_data["reduced_rank"] == 3
```

The option `-no-alexander` disables both Alexander stages. Use `-no-determinant` to disable only the modular prefilter. `-no-jones` and `-no-factors` control the other new stages independently. The Python API can remove the Jones ceilings with `None`. An explicit order passed to `khovanov_rank` is validated as a permutation and disables factorization so that the requested order is obeyed.

11.2 Budget and output qualifications

A time budget is cooperative. Validation, input conversion, heap construction, polynomial operations, and a large local morphism operation may delay a check. The object ceiling bounds one scanner complex's live objects, not all dictionary entries, all temporarily retained objects, morphism support, or total process memory. Use an external process limit when an application needs a hard deadline. A Jones filter's ceiling is different from the global ceiling: exhausting the former preserves completeness by falling back.

The output supplies rank counts and explanatory evidence, not a full homology basis or an independently checked formal proof. Its cube-degree histogram is not quantum-graded homology, and it does not compute integral torsion. Those omissions match the recognition-oriented purpose of the baseline. No use is made of the conjecture that Jones detects the unknot; equality with the unknot's Jones value always remains inconclusive.

11.3 Rebuilding the article and reproducing measurements

```
PYTHONHASHSEED=0 python -m unittest discover -s tests -v
python tools/benchmark.py --seconds 8 --repetitions 5
python tools/stress_followup.py --seconds 60
python tools/benchmark_tables.py
cd docs
pdflatex -interaction=nonstopmode -halt-on-error report.tex
pdflatex -interaction=nonstopmode -halt-on-error report.tex
```

The table generator reads the actual JSON measurements. It does not recompute expected speedups from hard-coded targets. The report source, generated table source, raw samples, baseline source, test log, and example generator remain in the distribution. A repeat run can produce different elapsed times without changing any mathematical result.

12 Conclusion

The strongest delivered result is a practical, exact, and tested improvement of the supplied program, together with a genuine factor-size bound for its full homology backend. Cheap scalar obstructions avoid expensive work on important Alexander-invisible examples; local algebra accelerates scans that still run; and connected-sum tensor products are handled as products of integer degree counts rather than exponentially large bases. The subroutine and restricted-class bounds are proved explicitly. A general quasi-polynomial implementation is not claimed, and the measured speedups, resource cutoffs, remaining worst case, and unverified large-instance comparisons are kept distinct.

References

- [1] The user-supplied `Knots.zip` archive, particularly `fast/fastunknot/`, `fast/tests/`, `fast/examples/`, `fast/results/`, and `docs/`. Input to this implementation study, 18 September 2026. Original code is preserved under `baseline_fastunknot/` in the companion distribution.
- [2] L. H. Kauffman, *State models and the Jones polynomial*, *Topology* **26** (1987), no. 3, 395–407. doi:10.1016/0040-9383(87)90009-7.
- [3] M. Khovanov, *A categorification of the Jones polynomial*, *Duke Mathematical Journal* **101** (2000), no. 3, 359–426. <https://arxiv.org/abs/math/9908171>.
- [4] D. Bar-Natan, *Fast Khovanov homology computations*, *Journal of Knot Theory and Its Ramifications* **16** (2007), no. 3, 243–255. <https://www.math.toronto.edu/drorbn/papers/FastKh/>.
- [5] P. B. Kronheimer and T. S. Mrowka, *Khovanov homology is an unknot-detector*, *Publications Mathématiques de l’IHÉS* **113** (2011), 97–208. <https://arxiv.org/abs/1005.4346>.
- [6] M. Lackenby, *Unknot recognition in quasi-polynomial time*, lecture slides, February 2021, 109 pages; especially pp. 13–14, 66–74, and 109. <https://people.maths.ox.ac.uk/lackenby/quasipolynomial-talk.pdf>.
- [7] M. Lackenby, *Incompressible surfaces, hierarchies and unknot recognition*, arXiv:2607.23350v1, 25 July 2026, 54 pages; especially Sections 9–10. <https://arxiv.org/abs/2607.23350>.